

Warehouse Robots

Weight:20%

Lecturer:Veronica Liesaputra

In automated multi-robot distribution centers (such as Amazon Robotics or Ocado), dozens of autonomous mobile robots share a constrained grid floor. Robots compete for available packages, must avoid physical collisions, and often block each other's paths in narrow corridors.

Designing an agent that performs well in isolation is not enough — your robot must navigate *crowded, competitive environments* where other agents are attempting to fulfil the same objectives.

Learning Objectives

By completing this assignment, you will:

1. Implement and evaluate rule-based and heuristic multi-agent strategies.
2. Work with a fixed-length numeric feature representation of an agent's state.
3. Apply a Genetic Algorithm (GA) to directly evolve the parameters of a percepts-to-action model through fitness-based selection, crossover, and mutation — without gradient descent or labelled training data.
4. Critically evaluate evolved behaviour against hand-designed heuristic agents, and analyse what a GA does and does not discover on its own.

1 Environment Rules & Setup

- **Grid Size:** 10×10 environment.
- **Actions (6 discrete choices):** 0: UP, 1: DOWN, 2: LEFT, 3: RIGHT, 4: PICKUP, 5: DISPATCH.
- **Episode length:** each episode runs for a maximum of 200 steps, or ends earlier once all packages have been delivered.
- **Collisions:** If two robots attempt to enter the same cell simultaneously, their moves fail and both remain in their current positions.
- **Scoring:** +1 point awarded upon successfully executing a DISPATCH action at a designated dispatch station while carrying a package.

1.1 State provided to agents

At each step, an agent function receives the current state of the world from its own point of view: its own grid position and carrying status, the positions and carrying status of any other robots, the locations of all remaining packages, and the locations of the fixed dispatch stations.

1.2 Provided starter files

You are given the following files — do not modify them:

- `warehouse_env.py` — the environment and step logic.
- `agents.py` — `RandomAgent` and utility functions (e.g., Manhattan distance).
- `feature_encoder.py` — converts the raw state into a fixed-length 215-value numeric feature vector: self-state, opponent state (positions and carrying status of other robots), a packages occupancy grid, a dispatch-station occupancy grid, a local crowding metric, and two derived boolean features (whether you are currently standing on a package cell, and whether you are standing on a dispatch cell). Use `encode_state(state)` from this file wherever a task calls for a numeric feature vector.

2 Tasks & Deliverables

2.1 Task 1: Baseline Evaluation & Failure Analysis (3 marks)

Build a `GreedyAgent` that calculates Manhattan distance to the nearest unclaimed package, moves directly to pick it up, and then routes to the nearest dispatch station.

Run your `GreedyAgent` against the provided `RandomAgent` over 10 fixed random seeds.

Deliverable:

1. Implementation of your `GreedyAgent` (`greedy_agent.py`).
2. Report the win rate of `GreedyAgent` vs. `RandomAgent`.
3. Write a one-paragraph failure analysis explaining why a deterministic greedy agent can still occasionally lose to a completely random opponent.

2.2 Task 2: Custom Heuristic Agent (4 marks)

Design and implement a modified agent (`SmartCustomAgent`).

Your modified agent must utilise at least one piece of contextual information ignored by your `GreedyAgent` (e.g., positions of competing robots, target package contestation, or local corridor congestion).

Deliverable:

1. Implementation of your `SmartCustomAgent` (`smart_custom_agent.py`).
2. Report the win rate of your modified agent against the standard `GreedyAgent` for 10 episodes.
3. Provide a brief description of your strategy and the reasons why you selected it.
4. Explain why your `SmartCustomAgent` can still occasionally lose to `GreedyAgent`.

2.3 Task 3: GA-Evolved Agent (7 marks)

You will evolve an agent’s behaviour directly using a genetic algorithm: the agent’s parameters are the chromosome, and fitness-based selection — not gradient descent — is what improves them across generations.

2.3.1 The model

Use the provided `feature_encoder.py` to convert each state into a 215-value feature vector $x = (x_1, \dots, x_{215})$. Implement a single-layer linear model — no hidden layer, no backpropagation — mapping this feature vector to the six actions:

$$v_j = w_{j,1}x_1 + w_{j,2}x_2 + \dots + w_{j,215}x_{215} + b_j, \quad j = 1, \dots, 6$$

for the six actions (UP, DOWN, LEFT, RIGHT, PICKUP, DISPATCH). The action taken is $\arg \max_j v_j$. The chromosome is the full set of weights and biases: 215×6 weights plus 6 biases, flattened into a single vector.

`GAAgent.__init__` must initialise this chromosome with random values. A freshly-initialised population must show no bias toward good behaviour — verify this by running your untrained population against `GreedyAgent` with no evolution applied. It should win roughly as often as it loses. If your untrained agents are already winning consistently, you have accidentally hard-coded intelligent behaviour into the model instead of leaving it to be learned by evolution.

2.3.2 GA requirements

You must implement, from scratch:

- `evalFitness(population)` — returns a fitness value per individual. A minimal starting point is packages delivered, but consider augmenting it (e.g., penalising collisions, rewarding successful `PICKUP/DISPATCH` attempts specifically, or accounting for episodes cut short by the 200-step limit). Justify your choice.
- `newGeneration(population, fitnesses)` — returns a new population of the same size, built from selection, crossover, and mutation of the old one. Elitism (copying top individuals unchanged into the next generation) is allowed.
- Selection, crossover, mutation — your choice of mechanism, but you must justify why your crossover operator makes sense given how the chromosome is structured. For example: does it make more sense to crossover at the level of individual weights, whole action-rows of weights (i.e., swapping an entire action’s decision function between parents), or some other grouping? A well-designed crossover should give two fit parents a good chance of producing a fit child — a naive element-wise crossover across a large flattened weight vector can easily destroy the coherent structure each parent evolved.
- A training schedule — a list of (opponent, number_of_generations) pairs specifying who your population trains against and for how long, e.g., `[("greedy", 300), ("self", 200)]` meaning 300 generations evaluated against `GreedyAgent`, followed by 200 generations of self-play against a frozen copy of an earlier generation’s best individual. Document what schedule you used and why you chose it.

Deliverable:

1. Your GA implementation (`ga_agent.py`).
2. Show evidence that a freshly initialised population shows no bias toward good behaviour prior to training (a short win-rate check against `GreedyAgent`, pre-evolution).

2.4 Task 4: Train, Evaluate, and Compare GAAgent (6 marks)

Run your GA for up to 500 generations following your declared training schedule. Record the average population fitness per generation — this is your primary evidence that evolution is working.

Take your final evolved population’s best individual (highest final fitness) and evaluate it head-to-head over 10 episodes against `GreedyAgent` and `SmartCustomAgent`.

Deliverable:

1. Implementation of your **GAAgent** training and testing (`train_and_evaluate.py`).
2. A plot of average (and best) fitness vs. generation number. You should see fitness trend upward over time, even if noisily. If it doesn't improve, that is itself a result worth reporting and diagnosing — don't hide a flat curve, explain what you think is going wrong instead.
3. Report win rates and total packages delivered for each match-up against **GreedyAgent** and **SmartCustomAgent**, the same way you did for the earlier agents.
4. Compare the performance of **GAAgent** with **GreedyAgent** and **SmartCustomAgent**, and discuss what the results indicate about the effectiveness of evolutionary search for this feature representation and action space.
5. Investigate whether **GAAgent** discovers behaviours that **GreedyAgent** does not exhibit. In particular, examine whether it learns behaviour resembling opponent awareness purely through evolutionary pressure, without any labelled examples. Analyse the evolved weights associated with the opponent-state features and discuss whether they are small (largely ignored) or large (actively influencing decisions).
6. Evaluate the sensitivity of the training process to the design of the fitness function. If you experimented with multiple fitness formulations, compare them and explain how they affected the evolved behaviour.

3 Files to Submit

- `greedy_agent.py` — Task 1
- `smart_custom_agent.py` — Task 2
- `ga_agent.py` — Task 3 (**GAAgent** class, `evalFitness`, `newGeneration`, and all selection/crossover/mutation helper functions)
- `train_and_evaluate.py` — Task 3 & 4 (runs the training schedule, produces the fitness-curve data, and runs the final evaluation matches against **GreedyAgent** and **SmartCustomAgent**)
- Written report (PDF)

Do NOT submit `warehouse_env.py`, `agents.py`, or `feature_encoder.py` — these are provided and will not be replaced by your versions when your code is run for marking.

Your code must run without errors. I will not have time to debug your program. Sticking to the framework provided is the best way to assure it will run on my machine.

Your code should contain all task implementations and evaluation loops.

Clean up the code source (as well as any debugging printing) before submission.

Screenshots of the text in your terminal and/or photos of hand-drawn diagrams are not the best way to add figures to your report. Figures and diagrams are great to have, but draw them properly (in Inkscape or PowerPoint for instance).

Finally, to pre-empt inevitable questions about the length of the report, let's say: "around 2000 words (which is roughly 4 pages)". But this is not an absolute number – more pages if you have lots of figures is not a problem.

4 Submission

This is an individual assignment. The percentage of the obtained marks out of 20 will be converted to a fraction of 20% that constitutes the weighting of this assignment in COSC343.

The assignment is due at **23:59 on Tuesday, 15 Sep 2026**. You should submit via Aoroa two files: `my_agent.tar.gz` containing your Python scripts, and a pdf file of your report. Don't zip these files into one attachment – add them separately to the submission.

Late submissions will incur a 5% penalty per day.

Academic Integrity and Academic Misconduct

Academic integrity means being honest in your studying and assessments. It is the basis for ethical decision-making and behaviour in an academic context. Academic integrity is informed by the values of honesty, trust, responsibility, fairness, respect and courage. Students are expected to be aware of, and act in accordance with, the University's Academic Integrity Policy.

Academic Misconduct, such as plagiarism or cheating, is a breach of Academic Integrity and is taken very seriously by the University. Types of misconduct include plagiarism, copying, unauthorised collaboration, taking unauthorised material into a test or exam, impersonation, and assisting someone else's misconduct. A more extensive list of the types of academic misconduct and associated processes and penalties is available in the University's Student Academic Misconduct Procedures.

Use of generative software such as ChatGPT is allowed as long as it is for the purpose of

aiding, not supplanting the effort of development code and/or improving the writing. If generative software is used, students must specify (in the report's appendix) how it was used and on what aspects of the assignment.

It is your responsibility to be aware of and use acceptable academic practices when completing your assessments. To access the information in the Academic Integrity Policy and learn more, please visit the [University's Academic Integrity website](#) or ask at the Student Learning Centre or Library. If you have any questions, ask your lecturer.

- [Academic Integrity Policy](#)
- [Student Academic Misconduct Procedures](#)